

RPC

- Stubs on either side for ease of use.
- At-most-one semantics (Exactly once on error). Use sequence number to detect duplicate requests. Reject old requests. Accept new requests.
- Grapevine as a distributed database. Store the following tables in the client and the server: Server: a) Server: IP address table; b) Export table -> service:dispatcher:uniq-server-id. Protects routines available through dispatchers. Unique service id used for restarts after crash -> rejected. c) Call Table: activity:last-seq-num:last-result. Activity => machine:pid.
- State per connection in server: Export Table, Call Table. One entry per service in the export table. One entry per client in the Call Table.
- Remembers the last 1 response. Detects duplicate calls, old calls and new calls using that. But cannot have multiple outstanding requests.
- Seq number just needs to be monotonically increasing, does not need to be sequential.
- Server crash: uuid changes, client needs to retry req. Client crash: Server does some useless work (from the old incarnation of the client). Note: Client needs to make sequence number as some function of time so that the new incarnation will have a greater sequence number > in call table.
- RPCs cannot be used with UNIX semantics for read/writes. This is because read/write with an fd would implicitly move the fd pointer causing the operation to be non-idempotent. The correct way to implement something like this would be to do something like a pwrite and pass along the particular offset that the operation must be performed.

Failures

- Reliability: Not doing the wrong thing. Availability: Doing right thing in specified amount of time. $Availability = (MTBF / (MTBF + MTTR))$.
- Infant morality bugs: New bugs that are slowly being discovered. Heisenbug: Non-deterministic, weird bugs. Bohrbugs: Deterministic, remove through testing.
- Software bugs: Wait for release cycle. Hardware bugs: Roll out ASAP.

Redundancy

- Redundancy fault tolerance — All 8 systems studied (Redis, ZooKeeper, Kafka, MongoDB, etc.) suffered data loss, corruption, or unavailability from a single fault on a single node, even with full replication in place.
- Two fault types: corruptions vs. I/O errors — Local file-system faults are either disk block corruptions (silent bad data returned) or latent sector errors (I/O error returned). How a system handles these two types differs sharply — don't conflate them.
- Undetected local faults → silent global corruption — When a node doesn't detect corruption (e.g. Cassandra with checksums off), it silently serves bad data to clients. The distributed layer never gets a signal to use a healthy replica. End-to-end integrity checks are essential.
- Crashing is the most common reaction — and it backfires — Systems most often crash on detecting a fault. But the fault is persistent (not a transient crash), so the node stays down, redundancy is reduced, and the cluster risks unavailability — without ever using healthy replicas to recover.
- Crash handling and corruption handling are entangled — Systems like Kafka use checksums to infer whether data was lost in a crash, but a disk corruption also triggers a checksum mismatch. The result: committed data gets truncated as if it were a crash artifact, causing real data loss.
- Local behavior has global consequences — and they interact unsafely — A locally truncated log (Kafka example) can allow the faulted node to be elected leader, causing cluster-wide data loss. Local recovery actions must be designed with the distributed protocol in mind — they can't be treated independently.

NFS

- All use RPCs.
- NFSv2 – NVRAM write buffer; uses a reply cache for the non-idempotent (mkdir) operation in case of duplicates; getattr overwhelming due to need to check cache coherency.
- NFSv3 – async-writes (can lose data that was written to server) -> write verified filled in by both client and server useful for restarts; 64-bit file size support; getattr less number -> any call returns attr (file based calls); pre-op and post-op time for write to large files for better cache coherency – do not need throw away entire file if none else has written.
- 2 concurrent client writing to the same file can be intermixed on the server – no version gives implicit locking. Even NFSv4 provides mechanisms for locking, in build but does not turn it on by default.
- NFSv4 – Compound statements - can interleave, not a transaction, complexity-failure in the middle; Open and Close – file locking(windows required), consistency, performance; Delegations - Performance improvement;
- NFSv4 can keep a future open() local to the client itself with delegations (read-only/exclusive).
- How to know if a delegation has expired – callback, lease period expiration; Adv-avoid getattr() keep ops local.
- Delegations are performance improvements, correctness X -> recovery is optimistic.
- NFSv4 client crashes – verifier id changes – release all locks. (lease expires also, release locks).
- All can keep writes buffered locally until close(). However, only NFSv4 can keep it buffered locally after close.

AFS

- Whole file caching; Prototype to throw away. Problems: TestAuth, GetFileStat high; CPU load-pathname traversal, process per client-context switch, fork-overhead; load-imbalance.
- open-to-close() semantics; callbacks for cache coherency.
- +Scalability, +Clearer-consistency, -Server state.
- Lease term: v1-poll on every open => 0; v2-Callbacks => infinite.

CODA

- Pessimistic vs Optimistic replica control. First vs Second class replica. Server replication: Volume Storage Group.
- Hoarding: hoard-db-user set priorities, inf priority-ancestor directories. Emulation: Reintegration:

Leases + Distributed Global Snapshot

- **Why Ordering Matters:** Needed for mutual exclusion, dedup, RSM, snapshots, make, distributed debugging.
- **Physical Clocks Unreliable:** Clocks drift (~50ms skew in datacenter). Central timeserver = bottleneck. Can't adjust backwards.
- **Happened-Before ($\rightarrow$):** (1) same-process order, (2) send $\rightarrow$ receive, (3) transitive. Means **a could** causally affect b.
- **Concurrent ($\parallel$):** $a \parallel b$ if $a \not\rightarrow b$ and $b \not\rightarrow a$. Concurrency is **not** transitive.
- **Lamport Clocks:** Increment on every event (IR1). Send: attach timestamp. Receive: $C_j = \max(C_j, T_m) + 1$ (IR2).
- **Lamport Guarantee:** $a \rightarrow b \implies C(a) < C(b)$. Converse is **false** — can't infer causality from clock values alone.
- **Total Order:** Break ties with process ID: $a \rightarrow b$ if $C(a) < C(b)$, or equal clocks and $P_i < P_j$. Non-unique — concurrent ops can go in any order.

- **Vector Clocks:** Each process holds vector $[C_1..C_n]$; increments own entry locally, takes element-wise max on receive. $C(a) < C(b)$ iff $a \rightarrow b$; incomparable $a \parallel b$. Drawback: size scales with # processes.
- **Snapshot Goal:** Record global state (all process states + all channel states) without stopping the system to detect stable properties: deadlock, termination, token counts.
- **Consistent Cut:** A cut is consistent if: whenever event i is in the cut and $j \rightarrow i$, then j is also in the cut. Equivalently, no recorded receive without its corresponding send.
- **System Assumptions:** Channels are FIFO, error-free, with infinite buffers and finite delay. Channel state = messages sent but not yet received.
- **Chandy-Lamport Initiation:** Any process can start — records own state, sends a **marker** on all outgoing channels before any further messages.
- **Marker Receipt (first time):** Record own state; record incoming channel as **empty**; forward marker on all outgoing channels.
- **Marker Receipt (subsequent):** Record channel state as all messages received on that channel since own snapshot was taken.
- **Termination:** Snapshot complete when all processes and all channels have been recorded. Requires a separate collection step to assemble the global picture.
- **Recorded State S^* May Not Be Real:** S^* might never have actually occurred. However, S^* is reachable from the initial state, and the final state is reachable from S^* . Stable properties valid in S^* are still meaningful.
- **Why S^* Is Still Useful:** To reconcile S^* with a true logical cut, swap orderings of concurrent pre- and post-snapshot events — always valid since concurrent op ordering doesn't matter.

Replicated State Machine

- **Fault-Tolerance:** System continues operating correctly despite one or more component failures. Any component can fail — network, storage, hardware, software.
- **Failure Types:** Fail-stop = nodes go silent (power, hardware, software crash). Asynchronous network = messages lost, corrupted, reordered, or delayed. Byzantine = malicious nodes sending contradictory or corrupted messages.
- **Measuring Fault-Tolerance:** MTBF/MTTR are client-centric metrics. More useful is *t-fault-tolerant*: system operates correctly with up to t concurrent replica failures. Fail-stop needs $t+1$ replicas; Byzantine needs $2t+1$ replicas (majority must outvote t bad nodes).
- **Correctness = Responsive + Consistent:** System must remain available (timely replies) and consistent (obey semantic contract). Strongest level is **linearizability** — system appears as a single atomic server to all clients.
- **RSM / SMR Idea:** Model the service as a deterministic state machine. Replicate a correctly-ordered log of client commands across all replicas. Since all replicas start from the same state and are deterministic, they reach the same state after executing the same log.
- **State Machine Requirements:** Must be *deterministic* (no random choices; output fully determined by input sequence), *single-sequence* (concurrent requests serialized), and use *logical time* (no dependence on physical clocks or real-time sensors).
- **Why Replicate the Log, Not the State?** Commands are small; state can be huge (millions of records). Executing commands is cheap. Replicas only need to agree on the ordered log, not transfer full state.
- **Replica Coordination — Agreement (IC1/IC2):** Every non-faulty replica must receive every command (IC1), and if the transmitter is non-faulty, all replicas use its value (IC2). Achieved via reliable broadcast — either client broadcasts to all, or a designated leader does.
- **Replica Coordination — Ordering:** Requests tagged with unique identifiers; a request is *stable* once no lower-ID request can arrive later. Three methods: (1) Lamport logical clock timestamps $\langle T_p, ID_p \rangle$ with FIFO channels; (2) synchronized real-time clocks with known max skew — wait Δ seconds for stability; (3) replica-generated IDs via Paxos-like majority voting.
- **When Can You Weaken Agreement/Ordering?** Idempotent commands can execute multiple

times safely. Read-only commands don't modify state — can serve from one replica. Commutative commands (EPaxos) don't need strict ordering. Nil-external commands don't return results — just make durable and execute later.

- **Faulty Output Devices:** A single voter is a single point of failure. Instead, replicate outputs — send from all replicas. External clients handle multiple identical outputs. Internal clients co-located with a replica can use that replica's output directly (client fault = replica fault).
- **Faulty Clients:** Fail-stop client won't release locks. Byzantine client sends contradictory inputs. Solutions: (1) replicate the client and buffer until enough identical requests arrive; (2) defensive programming — isolate client memory regions, use timeouts to release held resources.
- **Reconfiguration:** When faults exceed t , remove faulty nodes and add new replicas. Configuration changes must be agreed upon via consensus and scheduled for a future log position so all replicas and clients learn the new config before it takes effect.
- **Summary:** Replication = ordering + deterministic execution. SMR reduces fault-tolerance to the problem of log consensus. Underlies Paxos, Raft, Byzantine replication, and transactional serializability.

HA-NFS + REMUS

- **HA-NFS vs Remus:** Both use primary/backup, but HA-NFS replicates at the service layer (shared disk) while Remus replicates at the VM/infrastructure layer (no shared storage needed).
- **Primary-Backup key insight:** Backup is *passive* — ordering and determinism are handled by the primary, which sends state changes rather than raw inputs.
- **Failure detection tradeoff:** Short heartbeat timeout risks false positives (backup takes over a live primary); long timeout means longer unavailability. HA-NFS uses 10s + 5s conservatively.
- **When to ACK the client:** ACKing before backup confirms = better latency but risks data loss; waiting for backup ACK = correct but high latency. Async/batched approaches trade durability for performance.
- **HA-NFS keys:** Dual-ported shared disks (one writer guarantee), AIXv3 journaling filesystem for crash consistency, NFS v2 stateless server simplifies recovery.
- **Journaling:** Commit = transaction written to log. Crash after commit → replay journal. Crash before commit → no update (atomic). Checkpoint periodically flushes to in-place disk locations.
- **HA-NFS takeover:** Backup replays AIX journal exactly like a local crash recovery — stateless NFS v2 means no in-memory state to reconstruct. Clients just retry timed-out RPCs.
- **Remus approach:** Replicates entire VM state asynchronously; buffers outgoing network output until the checkpoint is committed to backup (*speculative execution*).
- **Remus performance costs:** VM overhead, checkpoint pause + dirty page transfer, and network buffering latency. Worst for write/memory-intensive workloads (e.g., kernel builds); read/network-heavy workloads (e.g., SPECweb) fare better.
- **Checkpoint interval tradeoff:** Shorter = less data loss, smaller rollback window, but higher overhead and more frequent pauses. Longer = lower overhead but more exposure on failure.

Serverless

- **Cloud computing core value:** elastic provisioning, pay-as-you-go, no upfront commitment — shifts capital expenses to operational expenses and matches capacity to demand.
- **XaaS hierarchy:** SaaS (full apps), IaaS (VMs, billed per allocation), FaaS (functions, billed per invocation × time × memory) — each layer abstracts more away from the user.
- **Serverless = FaaS:** Users deploy stateless, bounded-time functions; cloud provider handles all provisioning, scaling (0 to many), and resource management automatically.
- **Virtualization evolution:** Bare metal → VMs (shared hardware) → Containers (shared OS) → Serverless (shared runtime) — each generation shares more and reduces user responsibility.
- **Internal FaaS execution model:** Load balancer routes requests to workers; admission controller reuses or creates sandboxes (containers); sandbox evictor frees cold ones under memory pressure; Linux

cgroups/CFS enforce resource limits.

- **Cold start problem:** Starting a new container requires initializing runtime, installing dependencies, etc. — typically 0.2 to a few seconds, a key serverless limitation.
- **Current billing is SLIM:** Static (same memory limit for all invocations), Linear (CPU tied to memory), Interactive-only (no discount for low-demand time) — simple for providers but not true pay-for-use.
- **SLIM is profitable but unfair:** Customers pay for reserved-but-unused headroom; small-input invocations overpay relative to actual resource consumption.
- **Nearly-PFU model:** Introduces CPU-cap, spot-CPU (not needed immediately), reserved-CPU, and preemptible-mem — decouples CPU and memory, adds urgency levels, allows lending idle resources.
- **Nearly-PFU billing formula:** $\text{Cost} = \text{reserved-CPU-time} \times C_r + \text{borrowed-CPU-time} \times C_s - \text{lent-CPU-time} \times C_s$ — borrowers pay for spot resources, lenders get discounts, provider revenue stays neutral.
- **Linux can't natively support CPU reservation:** CPU pinning gives exclusive access but blocks sharing; weighted sharing is sharing-friendly but gives incorrect reservations — neither alone works.
- **Leopard's kernel fix:** New `cpu.resv_cpuset` cgroup interface + modified CFS scheduler — grants highest priority on reserved CPUs, best-effort sharing on spot CPUs, without relying on fairness for isolation.
- **Leopard results:** Provider throughput up 2.3×; customer cost down 34% (interactive) and 59% (batch) — key takeaway is that billing models must be treated as core system design, not an afterthought.

Chain replication

- **Structure:** Nodes arranged $\text{head} \rightarrow S \rightarrow S \rightarrow \dots \rightarrow \text{tail}$; writes enter at head, propagate forward; reads served only by tail
- **Strong consistency:** Tail is the single commit point — all reads see only committed state, no stale reads
- **Pipelining:** Multiple writes flow through chain concurrently, distributing replication work across all nodes
- **Write latency:** $O(nL)$ where n = chain length, L = one-way link latency — worse than primary/backup's $O(L)$ for writes
- **Read latency:** $O(2L)$ regardless of chain length — faster than primary/backup which must wait for concurrent writes to finish
- **Idempotency:** queries (reads) are idempotent; updates are not — clients must track unacked ops and resend on timeout
- **Head fails:** master promotes next node to head, notifies clients; client resends any unacked update to new head
- **Tail fails:** master promotes predecessor to tail; new tail acks all previously seen-but-unacked updates to clients
- **Inner node S fails:** predecessor S gets new successor S ; S sends S only updates in its *sent list* (unacked updates), not full history
- **Sent list:** per-node list of updates forwarded but not yet acked by tail; pruned as acks propagate backward through chain
- **Adding nodes:** new node T added at tail; T sends T its full state + in-flight updates; T becomes tail only when histories match
- **Weak chain variant:** allows reads from any node — improves throughput for read-heavy workloads but sacrifices strong consistency (stale reads possible)
- **CRAQ extension:** achieves strong consistency + read-any-node by checking sent list — if non-empty (uncommitted updates pending), re-route to tail or stall until committed